设计模式代码实现

创建型模式

1. 简单工厂模式：工厂的创建方法必须是static的，创建产品的时候不允许出现这个new 直接用factory的创建方法，直接用类的创建方法就行不用单独创建一个对象

```
class SimpleFactory{
  public SimpleFactory(){}

  public static Product createProduct(String type){
    Product product;
    swich(type)
    case "jiucai":
      product = new ConcreteProduct1();
      break;
    case "meat":
      product = new ConcreteProduct2();
      break;
    default:
      product = null;
      break;
    return product;
  }
}

abstract class Product{}
class ConcreteProduct1 extends Product{}
class ConcreteProduct2 extends Product{}

public class Main{
  public static void main(String[] args){
    SimpleFactory factory = new SimpleFactory();
    Product product = factory.createProduct("jiucai");

  }
}
```

总结：简单工厂必须是static方法，而且注意一下调用的时候是类直接调用，工厂和抽象工厂的创建方法就不用是static的了

2. 工厂模式 不光产品是多个，工厂也是多个，一个工厂对应一个产品，一个工厂只能生产一种产品

```
public class Main{
  public static void main(String[] args){
    Factory factory = new ProductAFactory();
    Product product = factory.createProduct();
    product.info();
  }
}
abstract class Factory{
  public abstract Product createProduct();
}
class ProductAFactory extends Factory{
```

```
  @Override
  public Product createProduct(){
    return new ProductA();
  }
}
class ProductBFactory extends Factory{
  @Override
  public Product createProduct(){
    return new ProductB();
  }
}
abstract class Product{
  public abstract void info();
}
class ProductA extends Product{
  @Override
  public void info(){
    System.out.println("productA");
  }
}
class ProductB extends Product{
  @Override
  public void info(){
    System.out.println("productB");
  }
}
```

3. 抽象工厂模式：一个产品有多种加工厂家产生多种不同的产品，而产品的种类也不只有一种，工厂自然也有很多个，工厂对应一种生产方法，不同的产品对应不同的函数

```
public class Main{
  public static void main(String[] args){
    AbstractFactory abstractFactory = new FactoryA();
    AbstractProductA aba = abstractFactory.createProductA();
    AbstractProductB abb = abstractFactory.createProductB();
    aba.info();
    abb.info();
  }
}

abstract class AbstractFactory{
  public abstract AbstractProductA createProductA();
  public abstract AbstractProductB createProductB();
}
class FactoryA extends AbstractFactory{
  @Override
  public AbstractProductA createProductA(){
    return new ProductA1();
  }
  @Override
  public AbstractProductB createProductB(){
    return new ProductB1();
  }
}
class FactoryB extends AbstractFactory{
  @Override
  public AbstractProductA createProductA(){
```

```
    return new ProductA2();
  }
  @Override
  public AbstractProductB createProductB(){
    return new ProductB2();
  }
}
abstract class AbstractProductA{
  public abstract void info();
}
class ProductA1 extends AbstractProductA{
  ...
}
class ProductA2 extends AbstractProductA{
  ...
}
abstract class AbstractProductB{
    public abstract void info();
}
class ProductB1 extends AbstractProductB{
  ...
}
class ProductB2 extends AbstractProductB{
  ...
}
```

4. 单例模式：保证一个类只有一个实例，并提供一个访问他的全局访问点

```
public class Main{
  public static void main(String[] args){
    SinglePattern s1 = SinglePattern.getInstance();
    SinglePattern s2 = SinglePattern.getInstance();
    SinglePattern s3 = SinglePattern.getInstance();
  }
}
class SinglePattern{
  private static SinglePattern instance;
  private SinglePattern(){}
  public static SinglePattern getInstance(){
    if(instance == null){
      instance = new SinglePattern();
    }

    return instance;
  }
}
```

⚠️ 变量是private static 方法是public static 构造函数是私有的不允许别人调用

5. 原型模式：用原型实例制定创建对象的种类，并且通过复制这些原型创建新的对象

```
public class Main{
  public static void main(String[] args){
    PrototypeA pA = new PrototypeA(1,1.1);
    PrototypeA pA1 = pA.clone();
    System.out.println(pA1.id);
    System.out.println(pA1.price);
```

```
    }
  }

  interface Prototype{
    public abstract Prototype clone();
  }
  class PrototypeA implements Prototype{
    public int id;
    public double price;
    public PrototypeA(int id,double price){
      this.id = id;
      this.price = price;
    };

    @Override
    public PrototypeA clone(){
      return new PrototypeA(id,price);
    }
  }
```

6. 建造者模式：将一个复杂对象的构建与他的表示分离，使得同样的构建过程可以创建不同的表示

```
public class Main{
  public static void main(String[] args){
    Builder builder = new ConcreteBuilder();
    builder.buildPart();
    Product product = builder.getResult();
    product.show();
  }
}

class Product{
  List<String> parts;
  public void Add(String part){
    parts.add(part);
  }
  public void show(){
    System.out.print("Composed by:");
    for(String part:parts){
      System.out.print(part + " ");
    }
    System.out.println();
  }
}

abstract Builder{
  public abstract void buildPart();
  public abstract Product getResult();
}
class ConcreteBuilder extends Builder{
  Product product= new Product();
  @Override
  public void buildPart(){
    product.Add("A");
    product.Add("B");
```

```
    product.Add("C");
  }
  @Override
  public Product getResult(){
    return product;
  }
}
```

结构型模式

1. 适配器模式：将一个类的接口转换成客户希望的另外一个接口 为了从不兼容到兼容

   Adapter 继承自被适应的类 拥有去适应的类作为对象

```
public class Main{
  public static void main(String[] args){

    USB usb = new Adapter(new TYPEC);
    usb.Request();
  }
}

class TYPEC{
  public void SpecificRequest(){
    System.out.println("type_c charging");
  }
}
class USB{
  public void Request(){
    System.out.println("usb charging");
  }
}
class Adapter extends USB{
  private TYPEC typec = new TYPEC();
  public Adapter(TYPEC t){
    this.typec = t;
  }
  @Override
  public void Request(){
    typec.SpecificRequest()
  }
}
```

2. 桥接模式:将抽象部分和实现部分分离，是他们都可以独立地变化 主要是为了分离维度 多个维度进行扩展

```
public class Main{
  public static void main(String[] args){
    Product product = new ProductA();
    product.setName("zhangsan");
    product.setColor (new Red());
    product.Operation();
```

```
  }
}

abstract class Product{
  private String name;
  protected Color color;

  public void setName(String name){
    this.name = name;
  }
  public String getName(){
    return this.name;
  }
  public void setColor(Color color){
    this.color = color;
  }
  public Color getColor(){
    return this.color;
  }
  public abstract void Operation();

}
class ProductA extends Product{
  @Override
  public void Operation(){
    color.OperationImp(this.getName)
  }
}
interface Color{
  public void OperationImp(String name);
}
class Red implements Color{
  @Override
  public void OperationImp(String name){
    System.out.println(name+"red");
  }
}
```

两条腿走路 继承一个 实现一个 实现的是成员变量的接口

3. 组合模式:将对象组合成树型结构用来表示部分-整体的层次结构

```
import java.util.List;
import java.util.ArrayList;

public class Main{
 public class void main(String[] args){
   AbstractFile root  = new Folder("root");
   AbstractFile folderA = new Folder("A");
   AbstractFile fileB = new File("B");
   root.add(folderA);
   root.add(fileB);

   root.printName();
 }

 static void print(AbstractFile root){
   root.printName();
```

```
    List<AbstractFile> files = root.getChildren();
    for(AbstractFile file: files){
      print(file);
    }
  }
}

abstract class AbstractFile{
  protected String name;

  public void printName(){
    System.out.println(name);
  }

  public abstract boolean add(AbstractFile file);
  public abstract boolean remove(AbstractFile file);
  public abstract List<AbstractFile> getChildren();
}

class Folder extends AbstractFile{
  private List<AbstractFile> files = new ArrayList<>();
  public Folder(String name){
    this.name = name;
  }
  @Override
  public boolean add(AbstractFile file){

    return files.add(file);
  }
  @Override
  public boolean remove(AbstractFile file){

    return files.remove(file);
  }
  @Override
  public List<AbstractFile> getChildren(){
    return files;
  }
}

class File extends AbstractFile{
  public File(String name){
    this.name = name;
  }
  @Override
  public boolean add(AbstractFile file){
    return false;
  }
  @Override
  public boolean remove(AbstractFile file){
    return false;
  }
  @Override
  public List<AbstractFile> getChildren(){
    return null;
  }
}
```

component类由leaf和composite两个子类继承 leaf可能会是composite的成员变量

4. 装饰器模式： 动态的给一个对象添加一些额外的职责. 继承还有组合

```
public class Main{
  public static void main(String[] args){
    Person p = new Student("Zhang San");
    p.Operation();
    //"The duty of Zhang San is study"

    Decorator d = new DecoratorA(p);
    p.Operation();
    //"The duty of Zhang San is study"
    //"The duty of Zhang San is live"

    p.addOperation();
    //"Hello World"
  }
}

abstract class Person{
  protected String name;

  public abstract void Operation();
}

class Student extends Person{
  public Student(String name){
    this.name = name;
  }
  @Override
  public void Operation(){
    System.out.println("The duty of"+name+" is study");
  }
}

abstract class Decorator extends Person{
  protected Person person;
}
class DecoratorA extends Decorator{
  public DecoratorA(Person person){
    this.person = person;
  }
  @Override
  public void Operation(){
    person.Operation();
    System.out.println("The duty of"+person.name+"is live");
  }

  public void addOperation(){
    System.out.println("hello world");
  }
}
```

5. 外观模式

```
public class Main{
  public static void main(String[] args){
    Facade fc = new Facade();
    System.out.println(fc.prove());
  }
}
class SubFlow1{
  public boolean isTrue(){return true;}
}
class SubFlow2{
  public boolean isOk(){return true;}
}
class SubFlow3{
  public boolean isSafe(){return true;}
}
class Facade{
  SubFlow1 sf1 = new SubFlow1();
  SubFlow2 sf2 = new SubFlow2();
  SubFlow3 sf3 = new SubFlow3();
  public boolean prove(){
    return sf1.isTrue()&&sf2.isOk()&&sf3.isSafe();
  }
}
```

6. 享元模式

可以共享的成为内部状态，外部环境决定的是外部状态，用一个享元工厂来维护享元池

补充：hashmap怎么用

```
import java.util.HashMap;
public class Main{
  public static void main(String[] args){
    HashMap<String,Integer> map = new HashMap<>();
  }
}



map.put(k,v);
map.get(k);
if(map.containsKey(k))
map.remove(k);
map.size();
map.isEmpty();
for(String k:map.keySet()){

}
for(HashMap.Entry<String,Integer> entry : map.entrySet()){

}
for(Integer v : map.values()){

}
```

下面是享元模式的代码

```java
import java.util.HashMap;
import java.util.Map;
abstract class  Flyweight{
  public abstract void operation();
}
class SharedFlyweight extends Flyweight{
  private  String type;//类型属于内部变量
  public SharedFlyweight(String type){
    this.type = type;
  }
  void operation(int x,int y){
    System.out.println("type = "+type+"at (" + x + "," + y + ")")
  }
  @Override
  public void operation(){
    threw new UnsupportedOperationException("Share Flyweight needs extrinsic state (x,y)");
  }
}
class UnsharedFlyweight extends Flyweight{
  private int x;
  private int y;
  private ShareFlyweight shared;
  UnshareFlyweight(int x,int y,SharedFlyweight shared){
    this.x = x;
    this.y = y;
    this.shared = shared;
  }

  @Override
  public void operation(){
    shared.operation(x,y);
  }
}
class FlyweightFactory{
  private final Map<String,SharedFlyweight> pool = new HashMap<>();

  public SharedFlyweight getShared(String type){
    if(pool.containsKey(type)){
      return pool.get(type);
    }else{
      SharedFlyweight fw = new SharedFlyweight(type);
      pool.put(type,fw);
      return fw;
    }
  }

  public int sharedCount(){
    return pool.size();
  }
}
public class Main{
  public static void main(String[] args){
    FlyweightFactory factory = new FlyweightFactory();

    Flyweight fw1 = new UnsharedFlyweight(1,1,factory.getShared("A"));
    Flyweight fw2 = new UnsharedFlyweight(2,2,factory.getShared("B"));
    Flyweight fw3 = new UnsharedFlyweight(3,3,factory.getShared("C"));
```

```
    fw1.operation();
    fw2.operation();
    fw3.operation();

    System.out.println(factory.sharedCount());


  }
}
```

一部分shared 一部分unshared unshared是shared的成员变量

7. 代理模式

```
abstract class Subject{
  public abstract void request();
}
class RealSubject extends Subject{
  @Override
  public void request(){
    System.out.println("RealSubject: handling request");
  }
}
class Proxy extends Subject{
  private String role;
  private RealSubject realSubject;
  public Subject(String name){
    this.name = name;
    this.realSubject = new realSubject();
  }
  @Override
  public void request(){
        if (!"admin".equals(role)) {
            System.out.println("Proxy: access denied");
            return;
        }
        System.out.println("Proxy: before request (log/permission)");
        realSubject.request();
        System.out.println("Proxy: after request (log/metrics)");
  }
}
public class Main{
  public static void main(String[] args){
    Subject subject1 = new Proxy("guest");
    subject1.request();

    Subject subject2 = new Proxy("admin");
    subject2.request();
  }
}
```

继承父类拥有兄弟

适配器模式是继承自被适应的人，拥有去适应的人作为成员变量

装饰器模式是继承的同时是成员变量

桥接模式是拥有一个接口作为成员变量 继承自另外一个

组合是复杂的类里面有简单的类作为成员变量，但是这两个是继承自同一个父类

适配器和桥接都是继承一个 一个作为成员变量 这两种模式的目的是不同的，

适配器是为了适配兼容　桥接是为了分离维度降低耦合，让特征在两个维度能够单独扩展

例如颜色和大小 颜色和形状 不想让他们在一起这样子类太多了，就通过这种方法分离扩展的维度

继承一个 拥有一个

1. 适配器模式 继承被适配方，拥有去适配别人的一方
2. 装饰器模式 继承父类，拥有父类
3. 桥接模式 继承抽象类，拥有接口（抽象父类就有）
4. 组合模式 继承父类，复杂子类拥有简单子类
5. 享元模式 非共享子类继承父类 拥有共享子类
6. 代理模式 继承父类，拥有兄弟类
7. 外观模式最简单，但是没有继承一个拥有一个，就是封装一系列类，只有一系列的拥有

行为型模式

1. 策略模式

```
public class Main{
  public static void main(String[] args){
    Operation operation = new Operation(new Strategy1());
    System.out.println(operation.Op(2,3));
    operation.setStrategy(new Strategy2());
    System.out.println(operation.Op(2,3));
  }
}
class Operation{
  private Strategy strategy;
  public Operation(Strategy strategy){
    this.strategy = strategy;
  }
  public int Op(int a,int b){
    return strategy.Op(a,b);
  }
  public void setStrategy(Strategy strategy){
    this.strategy = strategy;
  }
}
abstract class Strategy{
  public abstract int Op(int a,int b);
}
class Strategy1 extends Strategy{
  @Override
  public int Op(int a,int b){
    return a+b;
  }
}
class Strategy2 extends Strategy{
  @Override
```

```
  public int Op(int a,int b){
    return a*b;
  }
}
```

2. 中介者模式:用一个中介对象封装一系列的对象交互

```
import java.util.List;
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
        Mediator mediator = new ConcreteMediator();
        Person p1 = new Person1("Zhang San",mediator);
        Person p2 = new Person1("Li Si",mediator);
        Person p3 = new Person1("Wang Wu",mediator);
        Person p4 = new Person1("Yao Kaifang",mediator);
        Person p5 = new Person1("Du Xinyao",mediator);
        p1.sendMessage("Li Si","Time to get up.");
        p4.sendMessage("Du Xinyao","I love you.");
    }
}
abstract class Mediator{
    public abstract void SendMessage(String to,String message);
    public abstract void Add(Person person);
}

class ConcreteMediator extends Mediator {
  List<Person> persons = new ArrayList<>();
  @Override
  public void Add(Person person){
    persons.add(person);
  }
  @Override
  public void SendMessage(String to,String message){
    for(int i = 0 ; i < persons.size() ;i++){
      Person p = persons.get(i);
      if(to.equals(p.name)){
        p.getMessage(message);
        break;
      }
    }
  }

}
abstract class Person{
  protected String name;
  protected Mediator mediator;
  public abstract void sendMessage(String to,String message);
  public abstract void getMessage(String message);
}

class Person1 extends Person{
  public Person1(String name,Mediator mediator){
    this.name = name;
    this.mediator = mediator;
    mediator.Add(this);
```

```
  }
  @Override
  public void sendMessage(String to,String message){
    mediator.SendMessage(to,message);
  }
  @Override
  public void getMessage(String message){
     System.out.println(name + " get message:" + message);
  }
}
```

3. 观察者模式

```
import java.util.ArrayList;
import java.util.List;

public class Main{
  public static void main(String[] args){
    Observer ob1 = new fan("wwwsaidthat",1);
    Observer ob2 = new fan("wwwheardthis",2);

    Uper up = new Uper("www");
    up.Add(ob1);
    up.Add(ob2);
    up.update("new work");
  }
}

abstract class Observer{
  protected String accountName;
  protected int id;
  public void notify(String name){
    System.out.println("the Uper whose name is" +name+"has updated");
  }
}

class fan extends Observer{
  public fan(String accountName,int id){
    this.accountName = accountName;
    this.id = id;
  }
}

class Uper{
  String name;
  List<Observer> fans = new ArrayList<>();
  List<String> works = new ArrayList<>();
  public Uper(String name){
    this.name = name;
  }
  public void Add(Observer ob){
    fans.add(ob);
  }
  public void Remove(Observer ob){
    fans.remove(ob);
  }
```

```
  public void update(String work){
    works.add(work);
    this.Notify();
  }
  public void Notify(){
    fans.forEach(fan->fan.notify(name));
  }
}
```

4. 状态模式

```
public class Main{
  public static void main(String[] args){
    State state = new Happy();
    Person xm = new Xiaoming(state);
    xm.operation();
    xm.setState(new Sad());
    xm.operation();
  }
}
abstract class Person{
  protected State state;
  abstract void operation();
  abstract void setState(State state);
}
class Xiaoming extends Person{
  public Xiaoming(State state){
    this.state = state;
  }
  public void setState(State state){
    this.state = state;
  }
  public void operation(){
    state.operate();
  }
}
abstract class State{
  public abstract void operate();
}
class Happy extends State{
  public void operate(){
    System.out.println("Happy");
  }
}
class Sad extends State{
  public void operate(){
    System.out.println("Sad");
  }
}
```

5. 模版方法模式

```
public class Main{
```

```
  public static void main(String[] args){
    Pattern a = new A(10);
    System.out.println(a.getter());
    a.setter(new A(1));
    System.out.println(a.getter());
  }
}
abstract class Pattern{
  public abstract Object getter();
  public abstract void setter(Object ob);
}

class A extends Pattern{
  private int a;
  public A(int a){
    this.a = a;
  }
  @Override
  public Object getter(){
    return a;
  }
  @Override
  public void setter(Object ob){
   A tmp = (A) ob;
    a = tmp.a;
  }
}
class B extends Pattern{
  ...
}
```

6. 命令模式

```
public class Main{
  public static void main(String[] args){
    Receiver receiver = new Receiver("close");
    Invoker invoker = new Invoker(receiver);
    invoker.operate("open");
    invoker.operate("changing");
    invoker.operate("close");

  }
}
class Invoker{
  private Receiver receiver;
  private Commander opencommander ;
  private Commander closecommander ;
  private Commander changecommander ;
  public Invoker(Receiver receiver){
    this.receiver = receiver;
    opencommander = new OpenCommander(receiver);
    closecommander = new CloseCommander(receiver);
    changecommander = new ChangechannelCommander(receiver);

  }
  public void operate(String state){
```

```
    switch(state){
      case "open":opencommander.controlReceiver();
        break;
      case "close":closecommander.controlReceiver();
        break;
      case "changing":changecommander.controlReceiver();
        break;
    }
  }
}
abstract class Commander{
  protected Receiver receiver;
  public abstract void controlReceiver();
}
class OpenCommander extends Commander{
  public OpenCommander(Receiver receiver){
    this.receiver = receiver;
  }
  @Override
  public void controlReceiver(){
   receiver.setState("open") ;
    receiver.showState();
  }
}
class CloseCommander extends Commander{
  public CloseCommander(Receiver receiver){
    this.receiver = receiver;
  }
  @Override
  public void controlReceiver(){
    receiver.setState("close") ;
    receiver.showState();
  }
}
class ChangechannelCommander extends Commander{
  public ChangechannelCommander(Receiver receiver){
    this.receiver = receiver;
  }
  @Override
  public void controlReceiver(){
    receiver.setState("changing") ;
    receiver.showState();
  }
}
class Receiver{
  private String state;
  public Receiver(String state){
    this.state = state;
  }
  public void setState(String state){
    this.state = state;
  }
  public void showState(){
    System.out.println("the state of Receiver is " + state);
  }
}
```

状态模式是将状态作为一个成员变量，state类作为父类，然后根据子类的不同改变算法

命令模式是invoker有command和receiver作为成员变量，command有receiver作为成员变量 invoker调用command的方法 command的方法对receiver进行改变

模版方法模式是为了让多个不同的类可能关系不大能够共享同一套模版方法进行扩展

中介和观察都是相当于存在一个中间联系人，观察是省略了，直接把通知的功能放在被观察的人的里面了

语法上需要注意的是如果想要用子类的方法，必须在抽象类里面定义抽象方法，才能对于抽象类的对象使用

switch 要写default要不然返回值会出错

不要少写；

以后统一

类名每个单词首字母大写 大驼峰

方法名 小驼峰

属性名 全都是小写的

java里面的单例模式判断的时候用instance！ = null

原型模式clone的父类的返回值是抽象类，子类重写的时候可以修改成子类作为返回值

用list和arraylist别忘了导入包

定义类的时候 抽象类 别忘了写class